Flight Departure Board Sorting - Pure Arrays Java Solution

1. Detailed Question

An airport maintains a digital departure board. Each flight has a flight number, destination, and departure time. Due to schedule updates, the flights are stored in random order. Write a Java program using pure arrays to sort all flights from earliest departure time to latest departure time.

Condition: Use only arrays. Do not use Collections, ArrayList, or built-in sorting methods.

2. Input Data

Flight No	Destination	Time
AI203	Delhi	14:30
6E501	Mumbai	09:30
UK808	Chennai	22:15
SG302	Hyderabad	06:00
AI101	Bangalore	12:30

3. Diagram Representation

```
BEFORE SORTING

Index:       0          1          2          3          4

Flight:     AI203      6E501      UK808      SG302      AI101
Place:      Delhi      Mumbai     Chennai    Hyderabad  Bangalore
Time:       14:30      09:30      22:15      06:00      12:30

Goal: arrange by time from smallest to largest.

AFTER SORTING

Index:       0          1          2          3          4

Flight:     SG302      6E501      AI101      AI203      UK808
Place:      Hyderabad  Mumbai     Bangalore  Delhi      Chennai
Time:       06:00      09:30      12:30      14:30      22:15
```

4. Important Idea

This problem uses **parallel arrays**. The flight number, destination, and departure time at the same index belong to one flight. So whenever we swap two departure times, we must also swap the corresponding flight numbers and destinations.

Example: AI203, Delhi, 14:30 must move together. If only the time is swapped, the flight details become incorrect.

5. Pseudo Algorithm

```
START

Create flightNo[] array
Create destination[] array
Create departureTime[] array

FOR i = 0 to n - 2
    FOR j = 0 to n - i - 2

        IF departureTime[j] > departureTime[j + 1]
```

```
            Swap departureTime[j] and departureTime[j + 1]
            Swap flightNo[j] and flightNo[j + 1]
            Swap destination[j] and destination[j + 1]

        END IF

    END FOR
END FOR

Display sorted flight departure board

STOP
```

6. Bubble Sort Dry Run

```
Initial:
14:30   09:30   22:15   06:00   12:30

Pass 1:
14:30 > 09:30  -> Swap
09:30   14:30   22:15   06:00   12:30

14:30 < 22:15  -> No Swap
09:30   14:30   22:15   06:00   12:30

22:15 > 06:00  -> Swap
09:30   14:30   06:00   22:15   12:30

22:15 > 12:30  -> Swap
09:30   14:30   06:00   12:30   22:15

Pass 2:
14:30 > 06:00  -> Swap
09:30   06:00   14:30   12:30   22:15

14:30 > 12:30  -> Swap
09:30   06:00   12:30   14:30   22:15

Pass 3:
09:30 > 06:00  -> Swap
06:00   09:30   12:30   14:30   22:15
```

7. Complete Java Solution

```
public class FlightDepartureBoard {

    public static void main(String[] args) {

        String[] flightNo = {"AI203", "6E501", "UK808", "SG302", "AI101"};
        String[] destination = {"Delhi", "Mumbai", "Chennai", "Hyderabad", "Bangalore"};

        // Store time in 24-hour format.
        // Example: 1430 means 14:30, 930 means 09:30.
        int[] departureTime = {1430, 930, 2215, 600, 1230};

        System.out.println("Before Sorting:");
        displayFlights(flightNo, destination, departureTime);

        // Bubble Sort based on departure time
        for (int i = 0; i < departureTime.length - 1; i++) {

            for (int j = 0; j < departureTime.length - i - 1; j++) {

                if (departureTime[j] > departureTime[j + 1]) {

                    // Swap departure time
                    int tempTime = departureTime[j];
                    departureTime[j] = departureTime[j + 1];
                    departureTime[j + 1] = tempTime;

                    // Swap flight number
                    String tempFlight = flightNo[j];
                    flightNo[j] = flightNo[j + 1];
                    flightNo[j + 1] = tempFlight;

                    // Swap destination
                    String tempDestination = destination[j];
                    destination[j] = destination[j + 1];
                    destination[j + 1] = tempDestination;
                }
            }
        }

        System.out.println("\nAfter Sorting By Departure Time:");
        displayFlights(flightNo, destination, departureTime);
    }

    public static void displayFlights(String[] flightNo,
                                      String[] destination,
                                      int[] departureTime) {

        System.out.println("-----------------------------------------");
        System.out.println("Flight No\tDestination\tTime");
        System.out.println("-----------------------------------------");

        for (int i = 0; i < flightNo.length; i++) {
            System.out.println(flightNo[i] + "\t\t"
                    + destination[i] + "\t\t"
                    + formatTime(departureTime[i]));
        }
    }

    public static String formatTime(int time) {

        int hour = time / 100;
        int minute = time % 100;

        String h;
        String m;

        if (hour < 10) {
            h = "0" + hour;
        } else {
            h = "" + hour;
        }

        if (minute < 10) {
```

```
            m = "0" + minute;
        } else {
            m = "" + minute;
        }

        return h + ":" + m;
    }
}
```

8. Output

```
Before Sorting:
-------------------------------------------
Flight No       Destination     Time
-------------------------------------------
AI203           Delhi           14:30
6E501           Mumbai          09:30
UK808           Chennai         22:15
SG302           Hyderabad       06:00
AI101           Bangalore       12:30

After Sorting By Departure Time:
-------------------------------------------
Flight No       Destination     Time
-------------------------------------------
SG302           Hyderabad       06:00
6E501           Mumbai          09:30
AI101           Bangalore       12:30
AI203           Delhi           14:30
UK808           Chennai         22:15
```

9. Complexity

Time Complexity: O(n^2), because Bubble Sort uses nested loops.

Space Complexity: O(1), because sorting is done inside the same arrays.